

Отчет по лабораторной работе №3

1. Цель и требования

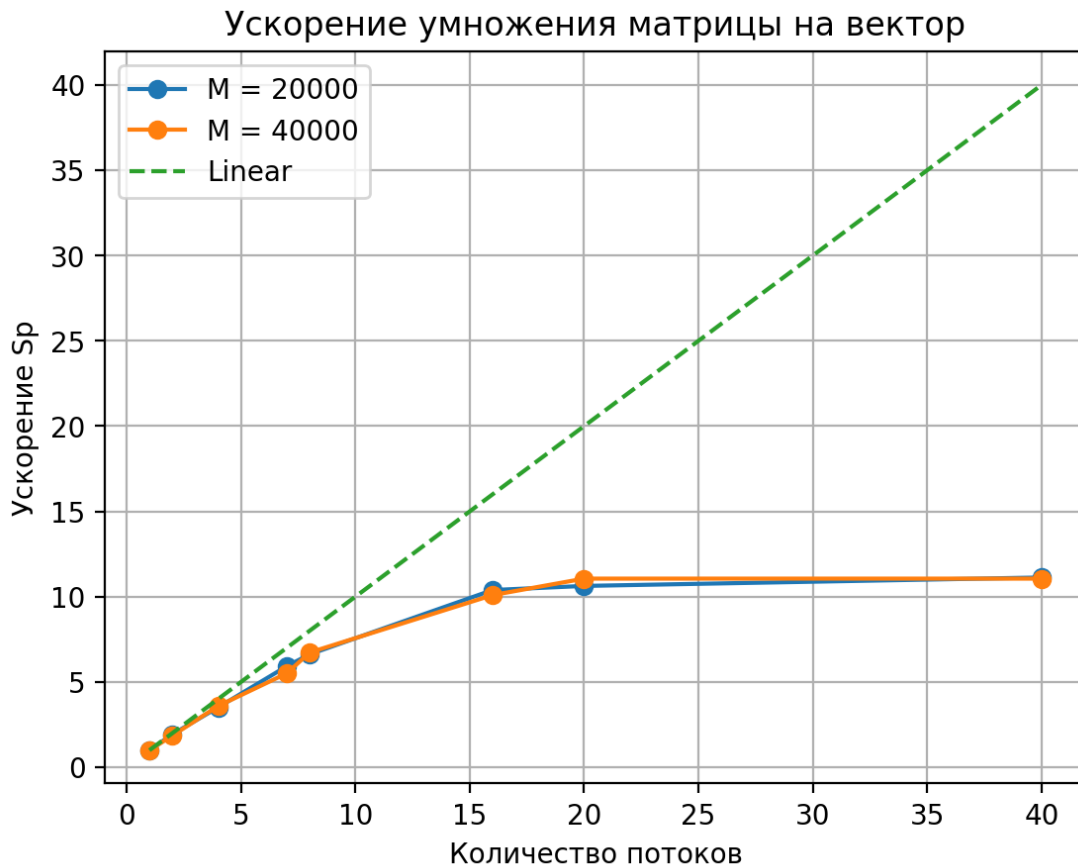
Цель работы - реализовать и проверить две многопоточные программы на C++: умножение матрицы на вектор и клиент-серверную систему задач.

Задание	Что требуется	Что реализовано
1	std::thread/std::jthread/async; параллельная инициализация; таблица и график ускорения для 1,2,4,7,8,16,20,40 потоков	Реализовано в matrix_vector_benchmark.cpp; результаты сохранены в task1_matrix_vector.csv
2	Сервер в отдельном потоке; очередь задач; контейнер результатов; 3 клиента; sin, sqrt, pow; проверка файлов	Реализовано в main.cpp, task_server.hpp и verify_results.cpp; результаты сохранены в client_*.csv

2. Задание 1. Умножение матрицы на вектор

Матрица хранится как одномерный массив double. Работа делится по строкам: каждый поток получает свой диапазон строк [begin, end). Поэтому потоки не записывают в одни и те же элементы result, matrix и vector.

Потоки	T, c M=20000	Sp M=20000	T, c M=40000	Sp M=40000
1	0.5833	1.00	2.2531	1.00
2	0.3017	1.93	1.2137	1.86
4	0.1662	3.51	0.6280	3.59
7	0.0985	5.92	0.4092	5.51
8	0.0881	6.62	0.3349	6.73
16	0.0561	10.39	0.2232	10.09
20	0.0549	10.63	0.2037	11.06
40	0.0524	11.13	0.2037	11.06



Вывод по масштабируемости:

- Ускорение растет почти линейно до 8 потоков: для $M=20000$ $Sp=6.62$, для $M=40000$ $Sp=6.73$.
- После 16-20 потоков рост замедляется: максимальное ускорение около 11 раз.
- На 40 потоках ускорение почти не увеличивается, потому что ограничением становится пропускная способность памяти, накладные расходы на создание потоков и конкуренция за кэш/память.
- Для большей матрицы $M=40000$ масштабирование немного стабильнее, так как вычислений больше относительно накладных расходов.

3. Итоговый вывод

В первой программе распараллеливание выполнено по строкам матрицы, поэтому гонки данных отсутствуют и mutex не требуется. Во второй программе общими ресурсами являются очередь задач, контейнер результатов, счетчик id и флаги состояния; они защищены mutex. condition_variable используется для ожидания задач и результатов без лишней нагрузки на процессор. Результаты соответствуют требованиям задания: есть таблица, график ускорения и проверка клиентских файлов.